Student Assessment Submission and Declaration

When submitting evidence for assessment, you must sign a declaration confirming that the work is your own.

Student name:	Pawel Mateusz Czerepski	ESL ID No	120071
		BSU ID No	575378
Submission date:			09.10.2026
Programme:	Computing foundations		
Module name and code:	Web development		
Title:	Assessment 2: multi device app		
Assessor name:	Sohail Ilyas		

Plagiarism

Plagiarism is a form of cheating. Plagiarism must be avoided at all costs and students who break the rules, however innocently, may be penalised. It is your responsibility to ensure that you understand correct referencing practices. As a university level student, you are expected to use appropriate references throughout and keep carefully detailed notes of all your sources of materials for material you have used in your work, including any material downloaded from the Internet. Please consult the relevant unit lecturer or your course tutor if you need any further advice.

Student declaration
I certify that the assignment submission is entirely my own work. I fully understand the consequences of plagiarism. I understand that making a false declaration is a form of malpractice.

Student signature:	Pawel Mateusz Czerepski	Date:	09.10.2026

Contents

Student Assessment Submission and Declaration 1
Personal Expense Tracker Documentation 2
Brief 2
Development Breakdown 3
Critical Reflection 4
Personal Development Reflections 5
References List 7
Appendix 1 8

Personal Expense Tracker Documentation

Brief

The project I chose to build is a personal expense tracker. The tracker will be designed as a single-page application with the front end built using ReactJS and the backend uses Express.js. The app design is simple, but it packs key robust features, such as allowing users to track their earned income and expenses over a given period. Once they have recorded their transactions, they have a live, simple dashboard where they can view their balances based on filters such as all, income, and expenses. The beauty of the app is that it allows the user to delete unwanted records. The app is also responsive to various device types owing to its responsive design and

adaptability with a user interface suited for both mobile and desktop users. The application also allows users to see their overall financial position without calculating it manually. This is made possible by the fact that the application brings income and expense records in one place. In this case, it is important to note that the application displays the balance, total income, and total expenses on the dashboard. This is critical in allowing users to review recorded transactions, thus giving them a better understanding and clarity about their overall financial position. These benefits serve as proof that the tracker is useful, not only for keeping financial records organized, but also for understanding how income and expenditure impact a user's available balance.

Development Breakdown

The development of this app factored in 3 key aspects. I used Express and Node.js to build the backend and applied a JavaScript array as transaction record storage. Instead of using complex helper functions, I simply applied standard JavaScript array methods, which ensured simplicity of the code and easy debugging (Express.js Documentation, 2024). The other critical aspect of the project is the React SPA Architecture using Vite (React Documentation, 2024). For instance, I used Vite to develop the frontend, since it is industrially recommended as a bootstrap for modern single-page applications like the expense tracking app. The adoption of Vite is in line with industry and university standards, as it ensures clear code that does not include unnecessary boilerplate files. I used basic vanilla CSS Grid to style the user interface (MDN Web Docs, 2024). The last aspect I considered was the design layout consideration, which I developed to ensure its interface is mobile first before moving on to the desktop layout. The choice of responsive design was influenced by user interaction with different devices. To achieve this, its

design was styled with CSS Grid in mind and media queries considering various breakpoints. On smaller screens, the elements are stacked vertically into one column to avoid cramping of elements, which makes the application page readable and user-friendly for users. On the other hand, the design layout on desktops allows the elements to occupy two columns for a clean page that is easy to navigate and record new transactions in the app. On desktop, the user interface is designed with proper flow to allow users to view and filter what they have added next to the transaction card.

Critical Reflection

I successfully developed the expense-tracking application, since it aligns with the main assessment needs in a direct manner. The use of Vite offered a smooth approach to the development, for it ensured minimal configuration overhead and fast build times through an efficient workflow. There is a reliable relationship between the Express backend and the React frontend, with instant updates to the dashboard during the addition or deletion of items. The use of basic Vanilla CSS helped me in developing a lightweight interface with no overhead for complex external CSS libraries. The application's scope is limited to recording income and expenses, viewing the balance, filtering transactions, and deleting records. The limited scope is a strength, as opposed to a weakness, because it keeps the interface straightforward, which makes it easy for users to understand how it functions. Also, designing the application to strictly support basic transaction tracking as opposed to making it a complete personal financial management system is critical to communicating its purpose clearly. Specifying these features allows users to know its purpose, which is important in informing their purchasing decisions.

The application was designed to work on mobile and desktop screens. This is evident in appendices 1 and 2, which show two different layouts. From the two appendices, the desktop version places the transaction form beside the transaction list, while the mobile version arranges the content vertically. These layouts are critical for both user experience and flexibility. This is because they allow the user to use the application from any device and anywhere.

Limitations: Some limitations in the developed versions include the loss of data for any transactions added once the Express server restarts or stops. Also, the application lacks login authentication or user accounts; hence, anybody can access the data or elements.

Future Improvements: It is crucial to use MongoDB or SQLite among other persistent databases instead of the in-memory array, for the purpose of enhancing the application for real-world use. This will ensure permanent saving of data.

Personal Development Reflections

The entire proceeds of developing the expense tracking application helped me to clearly understand the entire concept of full-stack development. I increased my familiarity and comfort with the use of Vite in building React SPAs. Also, I understand the idea that it only takes a few media queries and Vanilla CSS Grid to fully develop a responsive multi-device design. Developing the application enhanced my understanding of how frontend and backend work together. I also learned that developing a web application involves more than creating its visuals. This is because the designer must ensure that the frontend communicates with the backend to retrieve and update information. The development process has also helped me to recognize the

importance of data storage decisions. During this project, I learned that the way data is stored has a direct impact on the long-term usefulness of an application. This lesson was important because it helped me to understand and appreciate the importance of keeping the displayed information consistent with the data handled by the server.

References List

Express.js Documentation (2024) *Express: Fast, unopinionated, minimalist web framework for Node.js.* Available at: https://expressjs.com/ (Accessed: 10 October 2026).

MDN Web Docs (2024) *CSS Grid Layout.* Mozilla Developer Network. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout (Accessed: 10 October 2026).

React Documentation (2024) *State: A Component's Memory.* Meta Open Source. Available at: https://react.dev/learn/state-a-components-memory (Accessed: 10 October 2026).

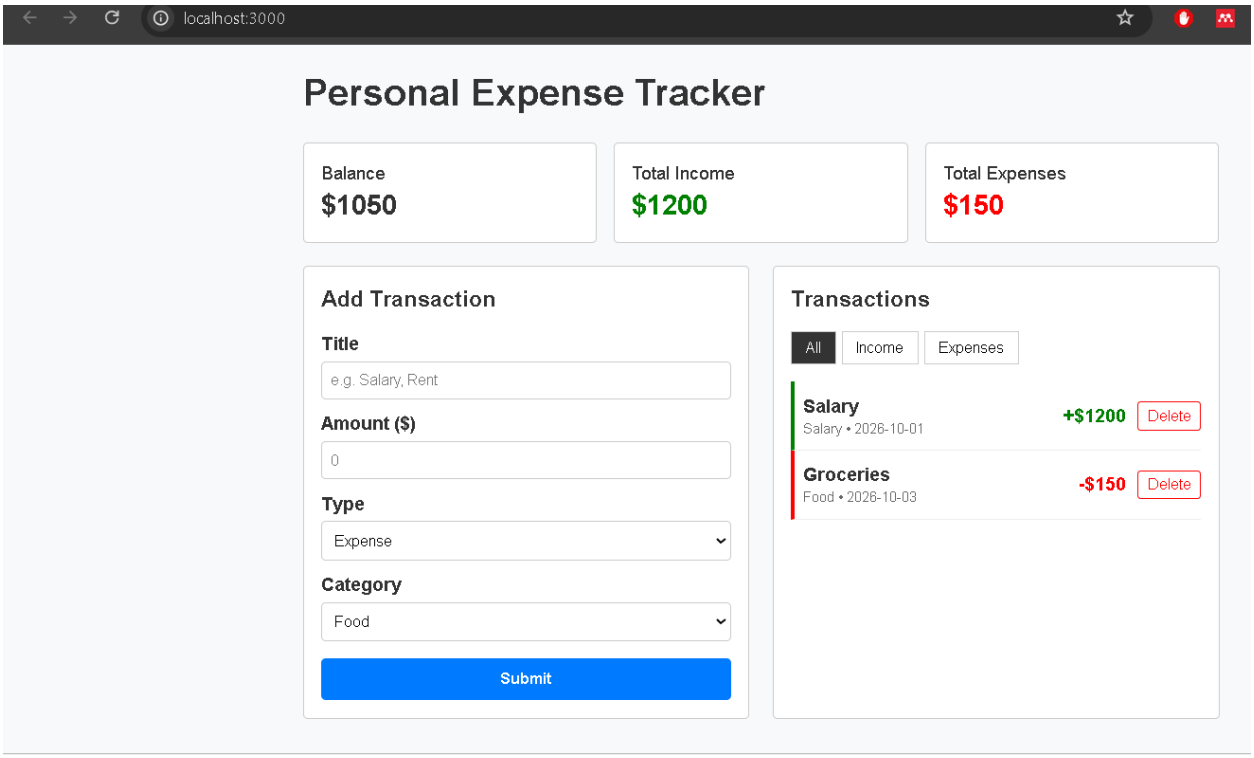
Vite Documentation (2024) *Vite: Next Generation Frontend Tooling.* Available at: https://vitejs.dev/ (Accessed: 10 October 2026).

Appendix 1

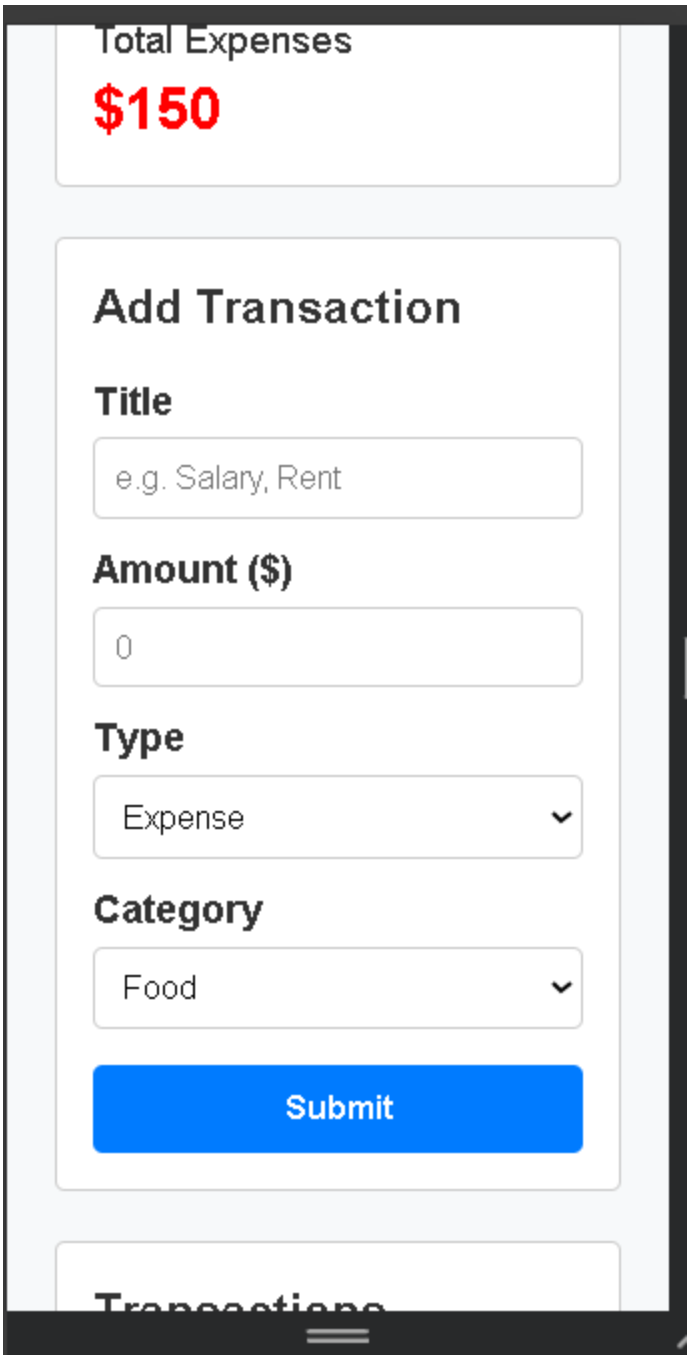
Github

walktrought

https://youtu.be/6_1qFW6TrX0



Appendix 2


Total Expenses
$150
Add Transaction
Title
e.g. Salary, Rent
Amount ($)
0
Type
Expense
Category
Food
Submit